

Create and Execute a Pre-Trigger in Azure Cosmos DB

Summary:-

In this lab, you will create a **pre-trigger** in Azure Cosmos DB using JavaScript and execute it from a .NET application. Unlike stored procedures, which are explicitly called by an application, triggers execute automatically before or after a document operation. In this lab, the pre-trigger adds default properties to an order document before it is stored in the container, ensuring consistent document structure.

Let's Start:-

After completing this lab, you will be able to:

- Understand the purpose of server-side programming in Azure Cosmos DB.
- Create a JavaScript pre-trigger inside a Cosmos DB container.
- Configure a .NET application to execute a pre-trigger.
- Automatically add default properties before a document is inserted.
- Verify that the trigger modifies the document before it is stored.

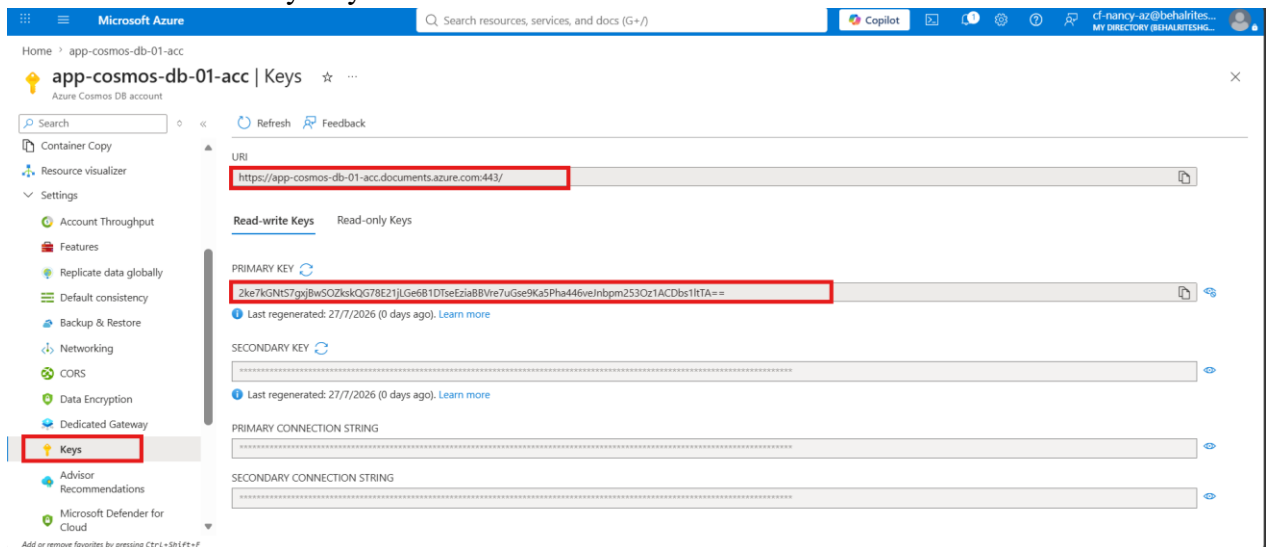
Lab Steps:-

Step 1 – Prepare the Azure Cosmos DB Container

- Sign in to the Azure portal. Create an Azure Cosmos DB account.
- Select **Azure Cosmos DB for NoSQL** from recommended APIs.
- Navigate to **Data Explorer**.
- Open the **orders-db** database and select the **Orders** container.
- Delete any existing documents from the container.
- **Reference:** This lab uses the .NET console application, Azure Cosmos DB account, database, and container created in the previous lab.
- For that **create** Azure Cosmos DB account by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure:
Workload type: Learning, **Region:** EAST US/ EAST US 2, **Capacity Mode:** Provisioned Throughput
- Enable **Apply Free Tier Discount**. Click **Next**.
- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Data Explorer**.
- Create a **Database** named **orders-db**.
- Inside the database, create a **Table** named **Orders** with
Partition Key: /customerId

- Open **Visual Studio Code**. Create a new project folder.
- Press **Ctrl + Shift + P** to open the **Command Palette**.
- Select **Show and Run Commands > .NET: New Project > Console App**(Rename it as cosmosdb-app) > **Default Directory > .slnx > Create Project**.
- **Open** the integrated terminal. Run:


```
cd .\storedProcedure\  
dotnet add package Microsoft.Azure.Cosmos  
dotnet add package Newtonsoft.Json
```
- Paste the given **Program.cs** code.
- Configure the following values in **Program.cs**:
 - Endpoint
 - Primary Key

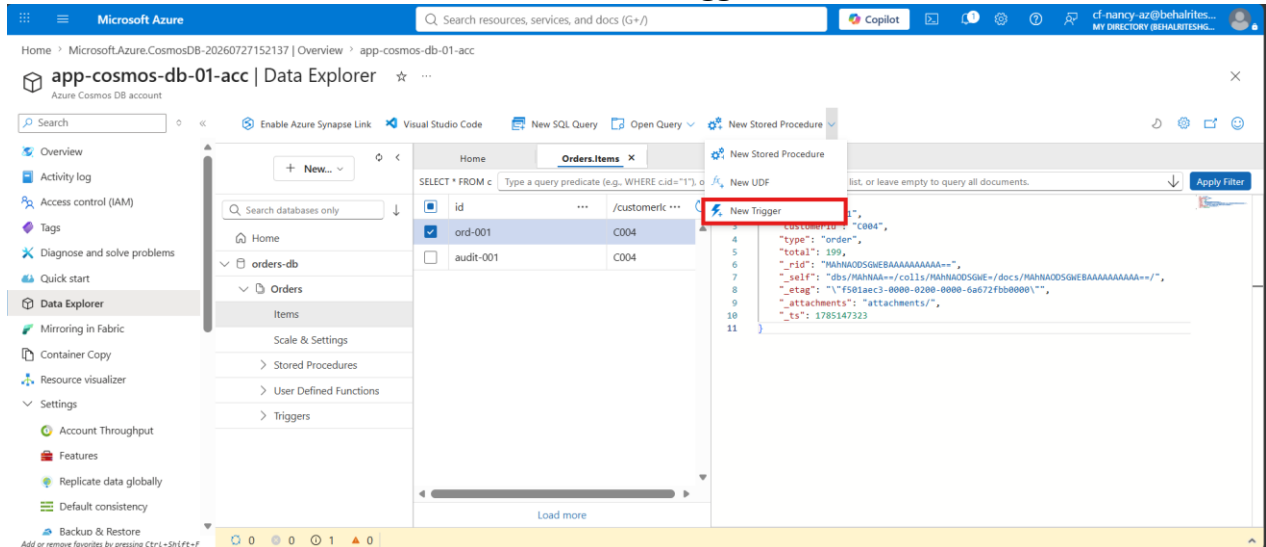


- Database ID: orders-db
 - Container ID: Orders
 - Partition Key: /customerId
- Open the following project files:
 - Program.cs**
 - storedProc-app.csproj**
 - storedProc-app.slnx**
- Replace the existing **Program.cs** code with the provided **Program2.cs** file.
- **Save** it and **Run** in terminal:
 - dotnet build
 - dotnet run
- Open the **Azure Portal**. Navigate to the **Azure Cosmos DB** account.
- Select **Data Explorer**. Open the **container** used in this lab. Refresh the **Items** node.
- **Verify** that:
 - One document contains the **Order** information.
 - One document contains the **Audit** information.

- This ensures that only the documents created during this lab are available for verification.

Step 2 – Create a New Trigger

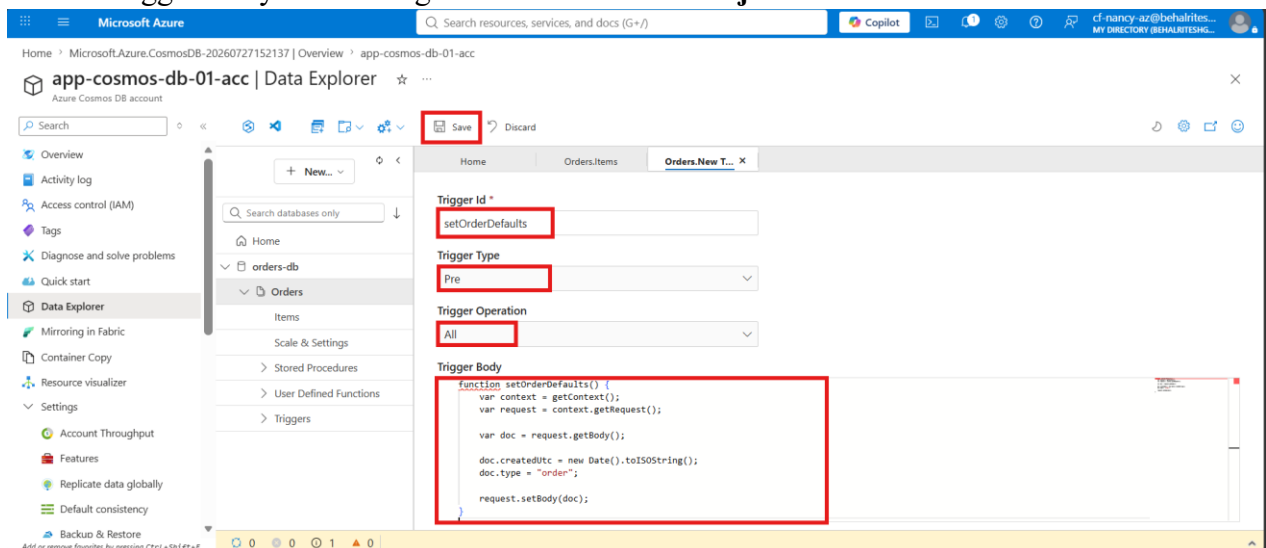
- Under the **Orders** container, expand **Scripts**.
- Select **New Stored Procedure**. Click **New Trigger**.



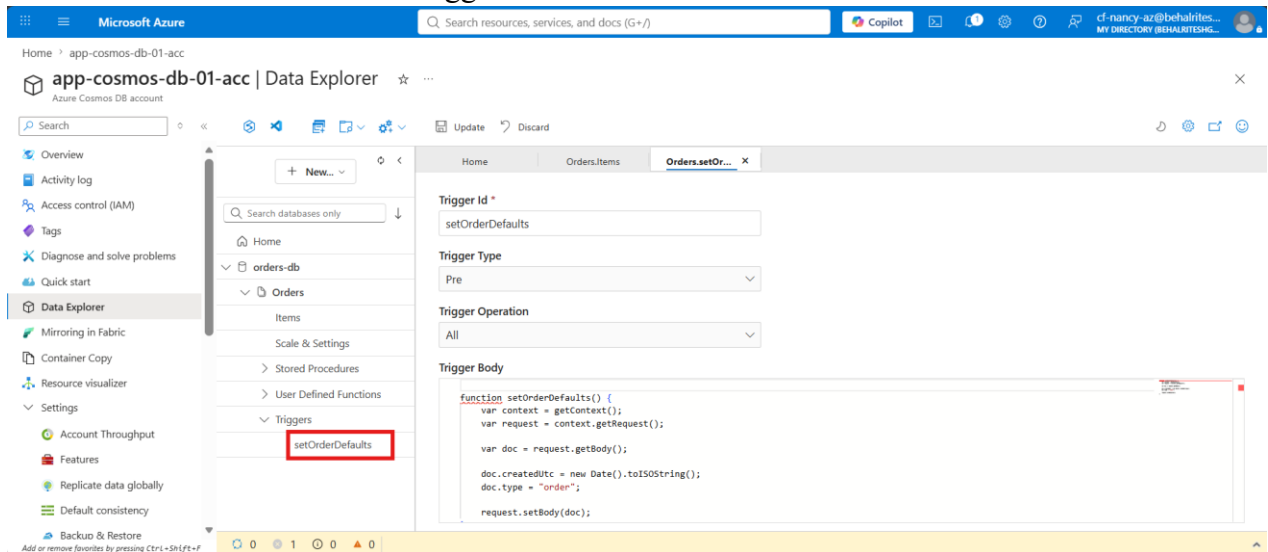
- This creates a server-side JavaScript trigger that will execute automatically whenever it is specified during document creation or replacement.

Step 3 – Configure the Trigger

- Enter the Trigger ID as: **setOrderDefaults**
- Trigger Type: **Pre**
- Trigger Operation: **All**
- Trigger Body: Paste the given **setOrderDefaults.js** code



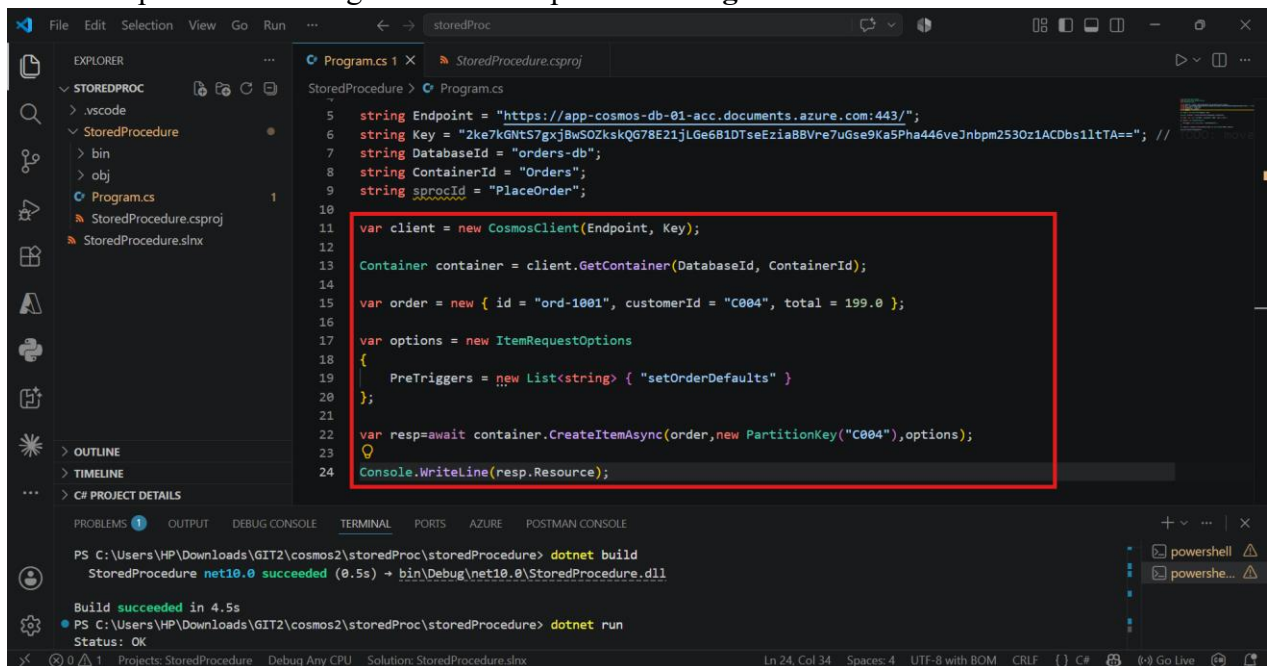
- Click **Save** to create the trigger.



- The pre-trigger is now configured to automatically validate and update documents before they are stored in the container.

Step 4 – Configure the .NET Application

- Open the .NET project in Visual Studio Code.
- Open the **Program.cs** file.
- Replace the existing code with the provided **Program.cs** code.

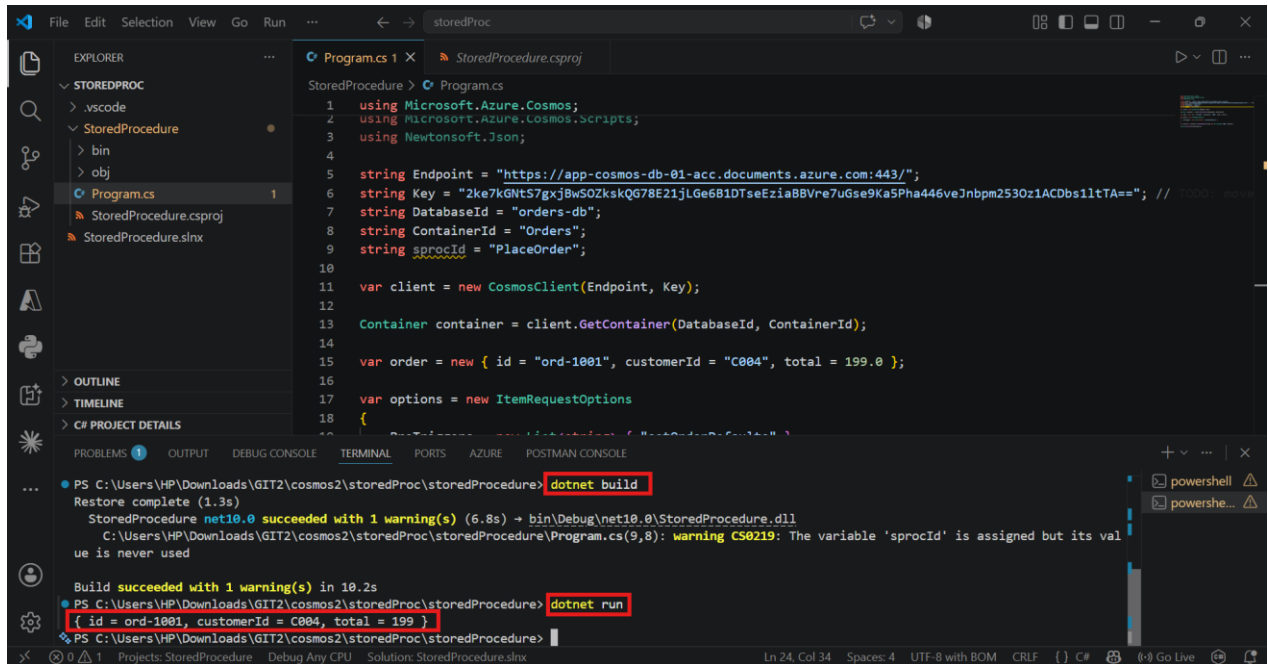


- Save the file.
- The application is configured to execute the **setOrderDefaults** pre-trigger while creating a new document.

- The pre-trigger runs before the document is written to the Azure Cosmos DB container, allowing default properties to be added automatically.

Step 5 – Build and Run the Application

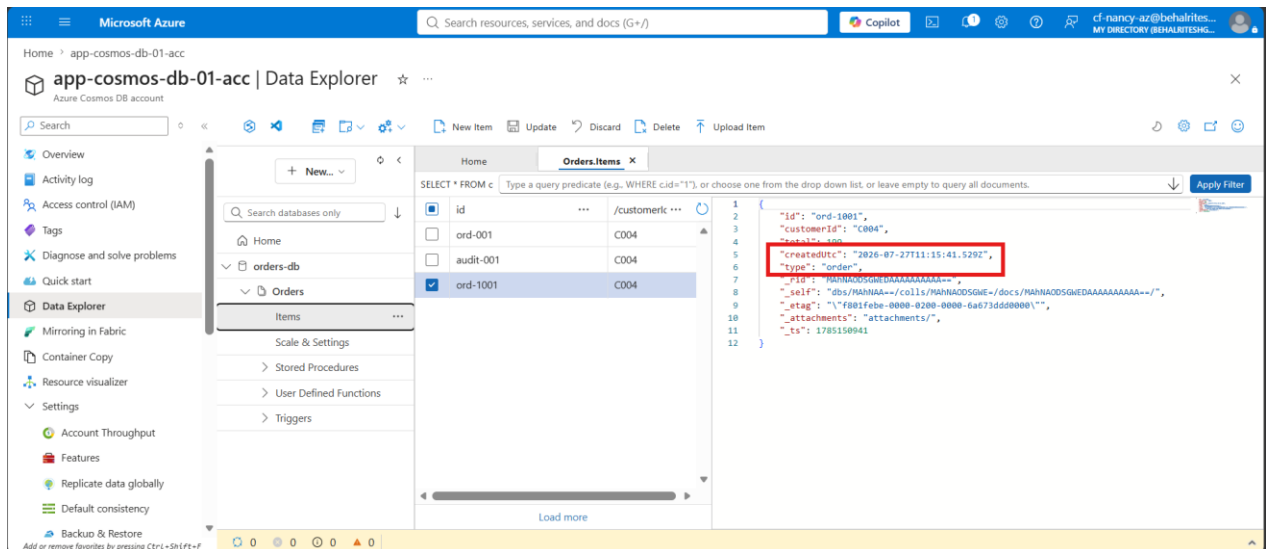
- **Run in terminal:**
 - dotnet build
 - dotnet run



- The application creates the document while executing the pre-trigger before the document is stored.

Step 6 – Verify the Stored Document

- Return to **Azure Portal**.
- Open **Data Explorer**.
- Refresh the **Orders** container. Open the newly created document.



- **Observe** that, in addition to the properties supplied by the .NET application, the document now also contains:
 - Type
 - CreatedDate
- These properties were automatically added by the pre-trigger before the document was stored.

Verification

Verify the following:

- The pre-trigger is successfully created.
- The .NET application executes successfully.
- The document is inserted into the Orders container.
- The document contains the original properties supplied by the application.
- The **Type** property is automatically added by the trigger.
- The **CreatedDate** property is automatically added by the trigger.
- The document is modified on the server before being stored in Azure Cosmos DB.